capital

# Custom Design Inspector Development

Software Version 2015.1

February 2016

© 2016 **Mentor Graphics Corporation**
**All rights reserved**.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

**RESTRICTED RIGHTS LEGEND 03/97**

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202-3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

**Contractor/manufacturer is:**
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

**TRADEMARKS:** The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/terms_conditions/trademarks.cfm.

**End-User License Agreement:** You can print a copy of the End-User License Agreement from: www.mentor.com/terms_conditions/enduser.cfm.

# Table Of Contents

**Introduction** ..... **2**
**Interfaces** ..... **5**
IXInspectionPanel ..... 5
IXIntegratorInspectionPanel ..... 6
IXTopologyInspectionPanel ..... 6
IXApplicationContext ..... 6
**Guidance** ..... **8**
**Example** ..... **9**

# Introduction

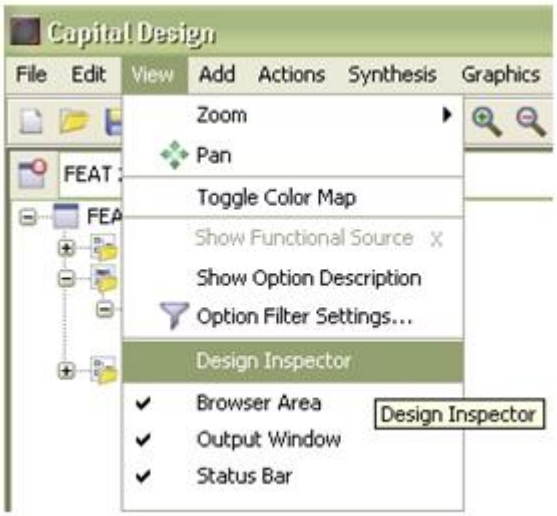
This document describes how to develop Custom Design Inspectors for Capital Topology or Capital Integrator. This type of plugin allows you to create custom views of the design (and selected parts of that design) allowing you to inspect design data in the format most suited to your needs.

This document has multiple references to Capital Integrator. All such references are also applicable to Capital Topology unless otherwise specified.

If you are **not** interested in developing Custom Design Inspectors, you do not have to read this document. If you are interested in developing Custom Design Inspectors, ensure that you have read and completed the exercises in the PluginDevelopment.pdf document before reading this document.

Henceforth we will refer to each instance of a Custom Design Inspector plugin that you may develop as an 'inspector'.

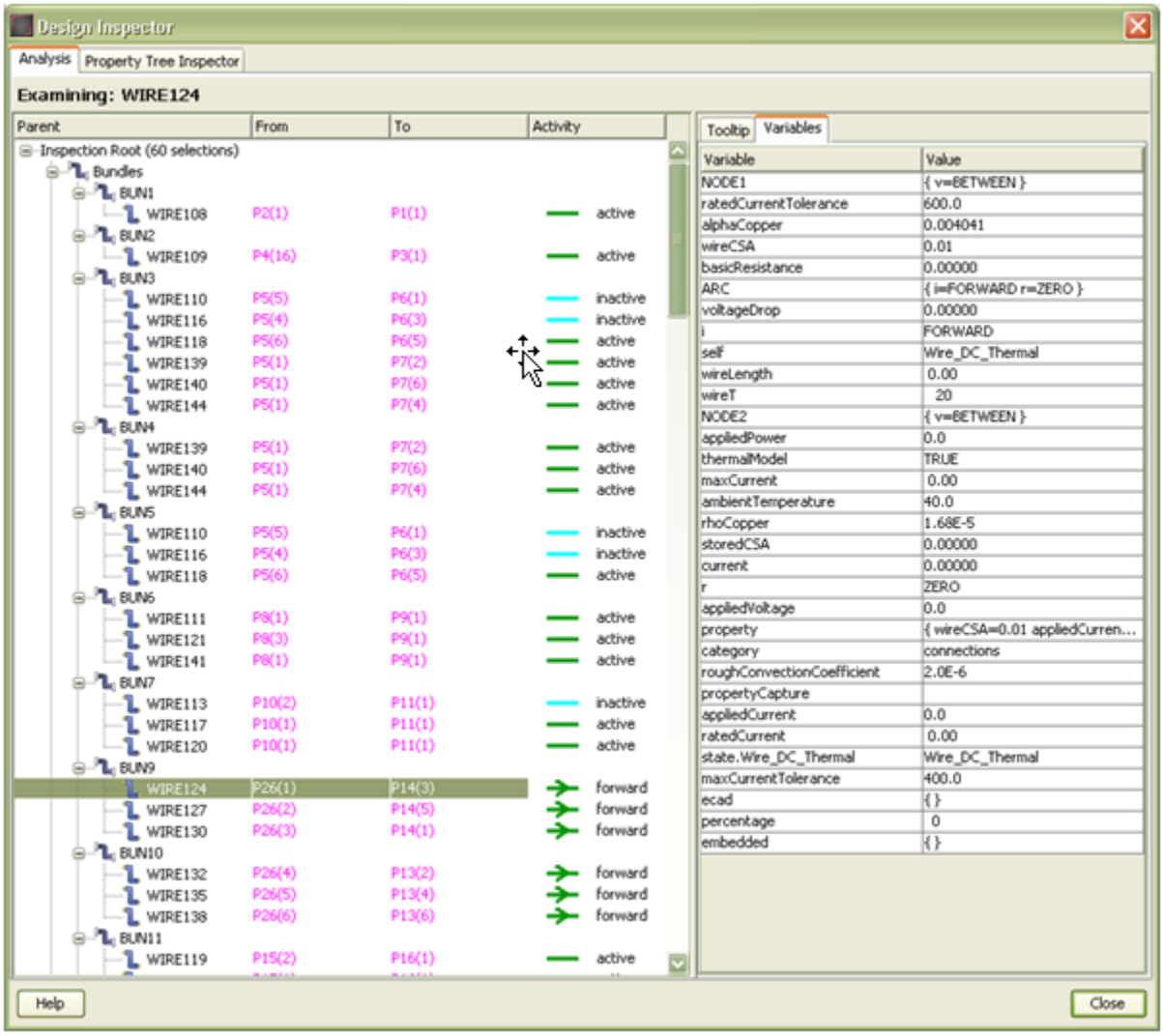
The inspectors can be seen by selecting 'Design Inspector' from the View Menu whilst an Integrator design is open. See below:



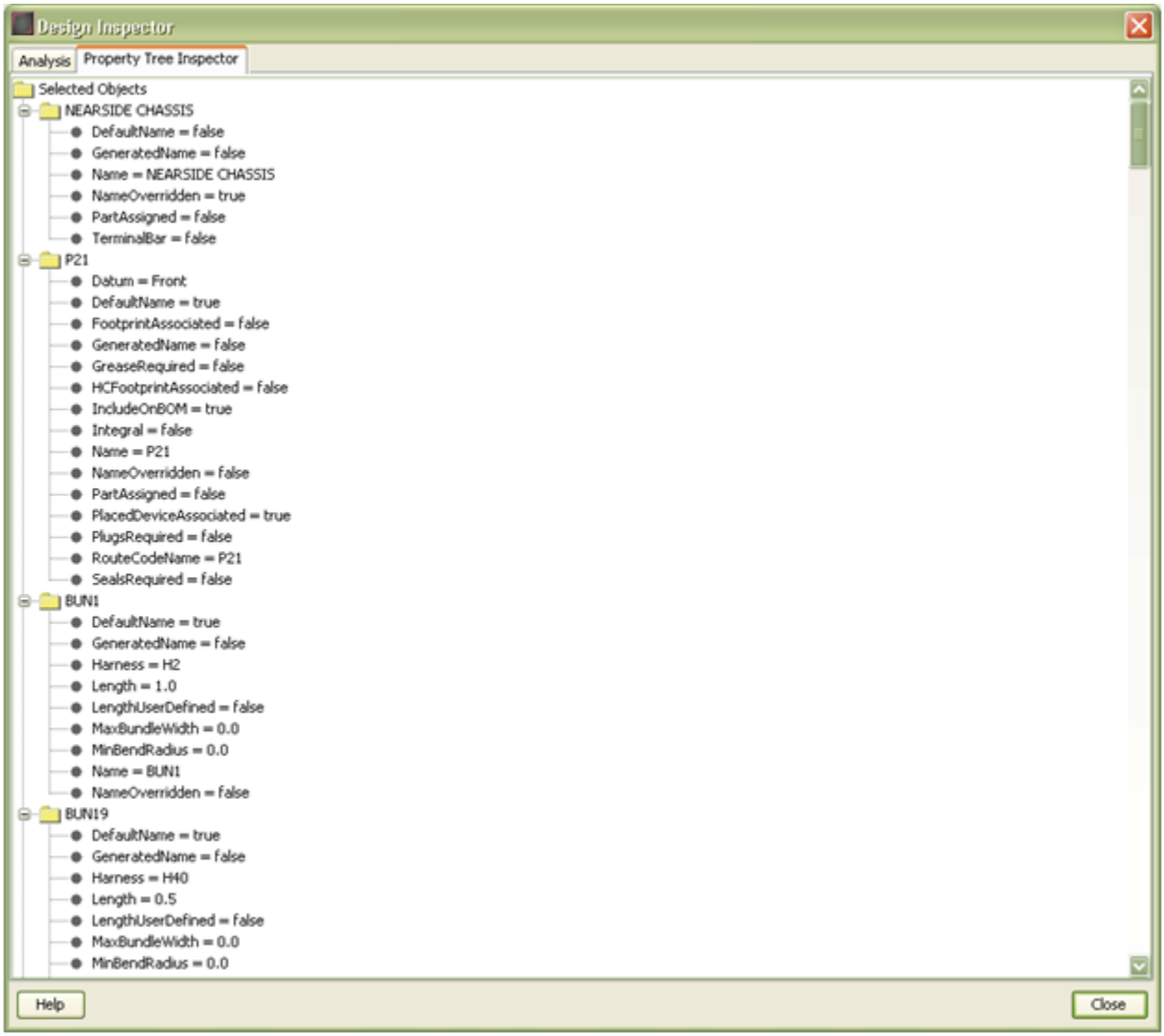

The Design Inspector will show all available inspectors when opened, whether an inspector is available or not is determined by the inspector itself.

Inspectors may show information for the whole design or the currently selected objects within the design. How and what the inspectors display to the user is completely under your control.

Examples of different inspectors are shown below:




The image above shows the Analysis design inspector during a simulation.




This shows the Property Tree Inspector that can be built from the example sources supplied and is described in more detail in the **Example** section of this document.

# Interfaces

## IXInspectionPanel

This interface contains the basic definition of an inspector. However, this interface should **not** be implemented directly, rather one of its child interfaces should be implemented so as to determine the product within which the inspector is to be available (i.e. implement **IXIntegratorInspectionPanel** if your inspector is to be used within Capital Integrator and implement **IXTopologyInspectionPanel** if your inspector is to be used within Capital Topology).

The **IXInspectionPanel** interface implements both the **IXPlugin** and **IXAvailability** interfaces. Hence you are required to implement all the methods defined in each of the interfaces ( *getName( ) , getVersion( ), getDescription( ) and isAvailable( IXApplicationContext )* – see PluginDevelopment.pdf for more information ).

Importantly, the *isAvailable( IXApplicationContext )* method is used to determine whether an inspector should be shown when the Design Inspector is opened. If an inspector determines that it is not available when the Design Inspector is opened it may not be shown until the Design Inspector is closed and re-opened.

The **IXInspectionPanel** interface is defined as follows:

```
public interface IXInspectionPanel extends IXPlugin, IXAvailability
{

        JPanel initialize( IXApplicationContext ctxt ) ;


        boolean update( IXApplicationContext ctxt ) ;


        void destroy( ) ;
}
```

*initialize( IXApplicationContext ctxt )* – This method must return a JPanel containing the user interface for the inspector. The user interface should be built from standard Java ‘Swing’ components and may be constructed in any way the user requires, to hold whatever content the user requires.

This method may query the **IXApplicationContext** instance for any information that may have a bearing on either the way the user interface is constructed or for any initial values that may be required to populate the user interface.

This method is messaged each time the Design Inspector is to be opened (so that the inspector is initialized correctly and the user interface ready to display). This is messaged whilst the Design Inspector is being opened and after the inspector has declared itself available for use in the *isAvailable( IXApplicationContext )* method.

*update( IXApplicationContext ctxt )* – This method requests that the inspector updates itself to reflect the current design data. This method allows the inspector to update the content of the user interface returned by the *initialize( IXApplicationContext )* method when the inspector is created.

This method is messaged often whilst the inspector is visible. Notably, this method will be messaged each time the selection is changed on a design that the inspector is viewing. Typically, most inspectors will wish to reflect only the contents of the current selection within their user interfaces (though this is not mandated), allowing the user to focus the inspection upon particular instances in the design.

*destroy( )* – This method is messaged to request the inspector to destroy itself as the Design Inspector is closing and the inspector is no longer required (and no longer visible to the user). Upon receipt of this message the inspector should release any resources it is holding and deconstruct the user interface it created in the initialize method.

This method will be messaged each time the Design Inspector is closed.

## IXIntegratorInspectionPanel

In order to implement an inspector that is to be used within Capital Integrator this interface must be implemented.

## IXTopologyInspectionPanel

In order to implement an inspector that is to be used within Capital Topology this interface must be implemented.

## IXApplicationContext

This interface allows the `isAvailable()` and `execute()` methods on the `IXAction` interface to make decisions or work on the objects returned by these methods:

`getApplication()` – this returns the current application. This allows a custom action to do slightly different things based on the application being used. The custom dialog that works with Capital Logic and Capital Integrator may require an extra tab in Capital Integrator, for example.

`getCurrentProject()` – this will return the current project the application is working on. Note that an application may have several projects open. This method will return the one that currently has focus.

`getCurrentDesign()` – this will return the current design the application is working on. Note that an application may have several designs open. This method will return the one that currently has focus.

`getSelectedObjects()` – this will return the set of `IXObjects` that are currently selected on the diagram. This method will never return null, but will return an empty set if no objects are selected.

`getLibrary()` – this method will return an `IXLibrary` object. This can be used by customers to search for library parts.

`getOutputWindow()` – returns an `IXOutputWindow` object that allows the custom action to display information to the CAF output window.

# Guidance

Here are some recommendations that will help you …

- In the *initialize( IXApplicationContext )* or *update( IXApplicationContext )* methods you may obtain a list of currently selected objects on the design by calling the *getSelectedObjects( )* method on the **IXApplicationContext** interface. This will allow you to populate the user interface only with the selected objects.

- After the *initialize( IXApplicationContext )* method has been messaged *update( IXApplicationContext)* is guaranteed to be messaged before the inspector is made visible. This ensures you can maintain a distinction between the code that builds the user interface (in *initialize*) and the code that populates the user interface with data (in *update*).

- You should ensure the *update( IXApplicationContext ctxt )* method runs quickly or else you may notice a slowdown in object selection performance whilst the Design Inspector is open.

- You can use the *getOutputWindow( )* method on the **IXApplicationContext** to obtain a means of sending messages to the console window. This is useful both during development and if you wish to display messages to the user during normal operation.

# Example

The following example shows how to build an inspector that shows the properties of each of the objects currently selected in the design.

This example makes use of a number of the methods available on the **IXApplicationContext**, both to write messages to the output window and to query the currently selected objects in the design.

You may also note how the inspector builds the user interface in the initialize method and simply updates the data displayed within it when the *update( IXApplicationContext )* method is messaged.

```
/**
 * Copyright 2007 Mentor Graphics Corporation. All Rights Reserved.
 * <p>
 * Recipients who obtain this code directly from Mentor Graphics use it solely
 * for internal purposes to serve as example plugin.
 * This code may not be used in a commercial distribution. Recipients may
 * duplicate the code provided that all notices are fully reproduced with
 * and remain in the code. No part of this code may be modified, reproduced,
 * translated, used, distributed, disclosed or provided to third parties
 * without the prior written consent of Mentor Graphics, except as expressly
 * authorized above.
 * <p>
 * THE CODE IS MADE AVAILABLE "AS IS" WITHOUT WARRANTY OR SUPPORT OF ANY KIND.
 * MENTOR GRAPHICS OFFERS NO EXPRESS OR IMPLIED WARRANTIES AND SPECIFICALLY
 * DISCLAIMS ANY WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE,
 * OR WARRANTY OF NON-INFRINGEMENT. IN NO EVENT SHALL MENTOR GRAPHICS OR ITS
 * LICENSORS BE LIABLE FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL
 * DAMAGES (INCLUDING LOST PROFITS OR SAVINGS) WHETHER BASED ON CONTRACT, TORT
 * OR ANY OTHER LEGAL THEORY, EVEN IF MENTOR GRAPHICS OR ITS LICENSORS HAVE BEEN
 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.
 * <p>
 */

package com.example.plugin.inspector;

import com.example.plugin.BasePlugin;
import com.mentor.chs.api.IXObject;
import com.mentor.chs.api.IXValue;
import com.mentor.chs.plugin.IXApplicationContext;
import com.mentor.chs.plugin.designinspection.IXIntegratorInspectionPanel;

import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTree;
import javax.swing.tree.DefaultMutableTreeNode;
import javax.swing.tree.DefaultTreeModel;
import java.awt.Color;
import java.awt.GridBagConstraints;
import java.awt.BorderLayout;

/**
 */
public class PropertyViewerTreeInspector extends BasePanel implements IXIntegratorInspectionPanel
{

protected DefaultMutableTreeNode root;
```

```
protected JTree tree;

protected DefaultTreeModel model;

public PropertyViewerTreeInspector()
    {
        super("Property Tree Inspector",
                "1.0",
                "Displays the properties of selected objects in a tree");
    }

protected void updateTable(IXApplicationContext applicationContext)
    {
        root.removeAllChildren();

        for (IXObject xObject : applicationContext.getSelectedObjects()) {

            // Create an node that is a child of the root node that holds the name of the object.
            DefaultMutableTreeNode object = new
            DefaultMutableTreeNode(BasePlugin.getObjectName(xObject));
            root.add(object);

            applicationContext.getOutputWindow().println(getName(), "   " + xObject.toHTML());

            // Foreach attribute, add nodes that display the attribute name & value and add them
            // as children of the 'object' node.
            for (IXValue value : xObject.getAttributes()) {
                if (!isEmpty(value.getValue())) {
                    DefaultMutableTreeNode attribute =
                            new DefaultMutableTreeNode(value.getName() + " = " + value.getValue());
                    object.add(attribute);
                }
            }

            // Foreach property, add nodes that display the property name & value and add them
            // as children of the 'object' node.
            for (IXValue value : xObject.getProperties()) {
                if (!isEmpty(value.getValue())) {
                    DefaultMutableTreeNode property =
                            new DefaultMutableTreeNode(value.getName() + " = " + value.getValue());
                    object.add(property);
                }
            }
        }

        model.reload();

        for (int i = 0; i < tree.getRowCount(); i++) {
            tree.expandRow(i);
        }

    }

public JPanel initialize(IXApplicationContext applicationContext)
    {
        panel.setLayout(new BorderLayout());

        // Create the root of the tree.
        root = new DefaultMutableTreeNode("Selected Objects");

        // Create the JTree.
        tree = new JTree();
        model = new DefaultTreeModel(root);
        tree.setModel(model);

        // Add the JTree to JScrollPane.
        JScrollPane pane = new JScrollPane(tree);

        // Add the JScrollPane to the  JDialog.
        panel.setBackground(Color.white);
        panel.add(pane, BorderLayout.CENTER);

        return panel;
    }
```

```
public boolean update(IXApplicationContext applicationContext)
    {
        applicationContext.getOutputWindow().println(getName(), "updating...");
        updateTable(applicationContext);
        tree.repaint();
        applicationContext.getOutputWindow().println(getName(), "...updated");
        return true;
    }
}
```

The above sample extends a **BasePanel** which implements the *isAvailable( IXApplicationContext )* method and the *destroy( )* methods (and those from **IXPlugin** via inheritance) described earlier, returning true to ensure this inspector is always visible.

The following is a listing of the **BasePanel** class

```
/**
 * Copyright 2007 Mentor Graphics Corporation. All Rights Reserved.
 * <p>
 * Recipients who obtain this code directly from Mentor Graphics use it solely
 * for internal purposes to serve as example plugin.
 * This code may not be used in a commercial distribution. Recipients may
 * duplicate the code provided that all notices are fully reproduced with
 * and remain in the code. No part of this code may be modified, reproduced,
 * translated, used, distributed, disclosed or provided to third parties
 * without the prior written consent of Mentor Graphics, except as expressly
 * authorized above.
 * <p>
 * THE CODE IS MADE AVAILABLE "AS IS" WITHOUT WARRANTY OR SUPPORT OF ANY KIND.
 * MENTOR GRAPHICS OFFERS NO EXPRESS OR IMPLIED WARRANTIES AND SPECIFICALLY
 * DISCLAIMS ANY WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE,
 * OR WARRANTY OF NON-INFRINGEMENT. IN NO EVENT SHALL MENTOR GRAPHICS OR ITS
 * LICENSORS BE LIABLE FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL
 * DAMAGES (INCLUDING LOST PROFITS OR SAVINGS) WHETHER BASED ON CONTRACT, TORT
 * OR ANY OTHER LEGAL THEORY, EVEN IF MENTOR GRAPHICS OR ITS LICENSORS HAVE BEEN
 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.
 * <p>
 */

package com.example.plugin.inspector;

import com.example.plugin.BasePlugin;
import com.mentor.chs.plugin.IXApplicationContext;

import javax.swing.JPanel;

/**
 */
public class BasePanel extends BasePlugin
{

protected JPanel panel;

  /**
 * Protected constructor used to ensure that the use of this class is via extension only.
 *
 * @param n - the name of the example custom action.
 * @param v - the version of the example custom action.
 * @param d - the description of the example custom action.
 */
protected BasePanel(String n, String v, String d)
    {
        super(n, v, d);
        panel = new JPanel();
    }

/**
 * Unlike IXAction.isAvailable() - this method should not return a value that is dependent on the
```

```
 * selection set. Inspectors dynamically redisplay data based on the selection set so they should
 * always be available to accept different selections.
 * <p/>
 *
 * @param context - the IXApplicationContext
 *
 * @return true
 */
public boolean isAvailable(IXApplicationContext context)
    {
        return true;
    }

public void destroy()
    {
        panel = null;
        panel = new JPanel();
    }
}
```